



Sri Eshwar
College of Engineering
An Autonomous Institution
Affiliated to Anna University, Chennai



LABORATORY RECORD

U23CS381 - APPLICATION DESIGN AND DEVELOPMENT



Sri Eshwar
College of Engineering
An Autonomous Institution
Affiliated to Anna University, Chennai



BONAFIDE CERTIFICATE

Certified that this is the Bonafide record of work done by,

Name: Mr. / Ms.

Register Number: of

I year B.E/B.Tech. in

U23CS381- APPLICATION DESIGN AND DEVELOPMENT during **I**

Semester of the Academic year **2023 – 2024 (Odd semester)**

Faculty In-Charge

Head of the Department

Submitted for the practical examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

INDEX

Sl. No	Date	Name of the Experiment	Page No	Mark	Signature
1		Static Website using HTML and CSS			
2		Form Validation using HTML, JavaScript			
3		Interactive Quiz using HTML, CSS, JavaScript			

4		Simple Animation using HTML, CSS, JavaScript			
5		Git & GitHub Operations			
6		File Management using Shell Scripting			
7		Mobile Application Development			
		a. Simple Calculator -MIT App Inventor			
		b. Step Counter -MIT App Inventor			
		c. Weather App-MIT App Inventor			
		d. Talk To Me- MIT App Inventor			
		e. Translation App -MIT App Inventor			
		f. To Do List- MIT App Inventor			
8		Poster Design using Canva			
9		Visual Infographic Design using Canva			
10		Game Development using Construct 2			
11		Images and Videos Editing using Blender			
12		Simple Animation using Blender			

Ex. No: 1	STATIC WEBSITE USING HTML, CSS

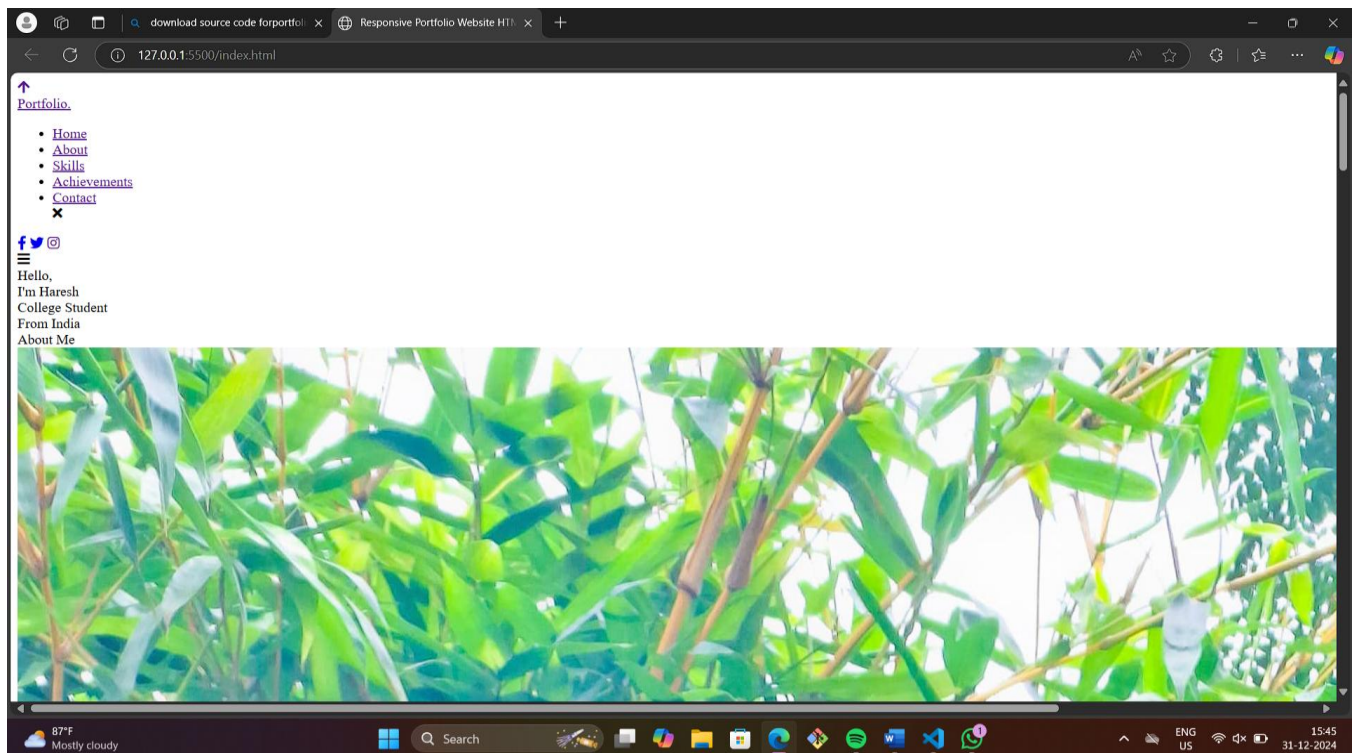
AIM

Develop a visually appealing static website with an intuitive user interface with multimedia content

- Add as text, images and videos to the webpage.
- Use CSS properties to make the form appealing.

CODE

OUTPUT



RESULT

Thus, the webpage has been created successfully.

Ex. No: 2	FORM VALIDATION USING HTML, JAVASCRIPT

AIM:

Create a simple form to collect the name, address, email ID and phone number from the user.

- Use CSS properties to make the form appealing.
- Write JavaScript functions for empty field validation.

CODE

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Simple Form Validation</title>
```

```
  <style>
```

```
    body {
```

```
      font-family: Arial, sans-serif;
```

```
      max-width: 400px;
```

```
      margin: 20px auto;
```

```
}
```

```
label,
```

```
input,
```

```
button {
```

```
    display: block;
```

```
    width: 100%;
```

```
    margin-bottom: 10px;
```

```
}
```

```
.error {
```

```
    color: red;
```

```
    font-size: 12px;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h3>Registration Form</h3>
```

```
<form id="registrationForm">
```

```
    <label for="name">Name:</label>
```

```
    <input type="text" id="name" name="name" required pattern="[A-Za-z ]{3,}" title="Name should be at least 3 characters long and only contain letters and spaces.">
```

```
    <span class="error" id="nameError"></span>
```

<label for="email">Gmail:</label>

<input type="email" id="email" name="email" required pattern="[a-zA-Z0-9._%+-]+@gmail\.com" title="Please enter a valid Gmail address.">

<label for="mobile">Mobile Number:</label>

<input type="tel" id="mobile" name="mobile" required pattern="[0-9]{10}" title="Mobile number must be 10 digits.">

<label for="password">Password:</label>

<input type="password" id="password" name="password" required minlength="6" title="Password must be at least 6 characters long.">

<label for="retypePassword">Retype Password:</label>

<input type="password" id="retypePassword" name="retypePassword" required>

<button type="submit">Submit</button>

</form>

<script>

const form = document.getElementById('registrationForm');

form.addEventListener('submit', function(event) {

const name = document.getElementById('name');

```
const email = document.getElementById('email');

const mobile = document.getElementById('mobile');

const password = document.getElementById('password');

const retypePassword = document.getElementById('retypePassword');


let isValid = true;


if (name.value.trim().length < 3) {

    document.getElementById('nameError').textContent = 'Name must be at least 3 characters.';

    isValid = false;

} else {

    document.getElementById('nameError').textContent = '';

}


if (!email.value.endsWith('@gmail.com')) {

    document.getElementById('emailError').textContent = 'Only Gmail addresses are allowed.';

    isValid = false;

} else {

    document.getElementById('emailError').textContent = '';

}


if (!/^\d{10}$/.test(mobile.value)) {

    document.getElementById('mobileError').textContent = 'Mobile number must be 10 digits.';

    isValid = false;

} else {

    document.getElementById('mobileError').textContent = '';

}
```



```
}
```

```
if (password.value.length < 6) {
```

```
    document.getElementById('passwordError').textContent = 'Password must be at least 6 characters long';
```

OUTPUT

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:5500/index.html'. The browser has several tabs open, including 'download source code for portfolio', 'ChatGPT | OpenAI', 'ChatGPT', and 'Simple Form Validation'. The main content area displays a 'Registration Form' with the following fields and values:

- Name: my name
- Gmail: fthdrg
- Mobile Number: 3434532454542
- Password: *****
- Retype Password: *****

Below the fields is a 'Submit' button. The Windows taskbar is visible at the bottom, showing the Start button, Search bar, and various application icons. The system tray on the right indicates the language is 'ENG US', the time is '15:51', and the date is '31-12-2024'.

RESULT

Thus, the webpage has been created successfully.

| | |
|------------------|---|
| Ex. No: 3 | INTERACTIVE QUIZ USING HTML, CSS, JAVASCRIPT |
| | |

AIM:

Create an interactive quiz that allows the user to select answers to multiple-choice questions.

- Use CSS properties to make the form appealing.
- Write JavaScript functions for calculation the user's score and provide feedback based on their performance.

CODE

This screenshot shows the initial state of the VS Code editor. The Explorer panel on the left displays a file tree for a project named 'PORTFOLIO', containing an 'images' folder and several image files. The main editor window is open to 'index.html', showing the following code:

```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Interactive Quiz</title>
8   <style>
9     body {
10       font-family: Arial, sans-serif;
11       margin: 20px;
12     }
13
14     .quiz-container {
15       max-width: 600px;
16       margin: auto;
17     }
18
19     .question {
20       margin-bottom: 20px;
21     }
22
23     .answers button {
24       display: block;
25       margin: 5px 0;
26       padding: 10px;
27       background-color: #007bff;
28       color: white;
29       border: none;
30       border-radius: 5px;
31       cursor: pointer;
32     }
33
34     .answers button:hover {
35       background-color: #0056b3;
36     }
37
```

The status bar at the bottom indicates the cursor is at line 18, column 9, with 4 spaces, UTF-8 encoding, CRLF line endings, and the HTML document type. The port 5500 is open, and Prettier is configured for formatting.

This screenshot shows the completed code in the VS Code editor. The 'index.html' file now includes the following structure:

```
2 <html lang="en">
4 <head>
8   <style>
37
38     #result {
39       font-size: 20px;
40       font-weight: bold;
41       margin-top: 20px;
42     }
43   </style>
44 </head>
45
46 <body>
47   <div class="quiz-container">
48     <h1>Interactive Quiz</h1>
49     <div id="quiz">
50       <div class="question" id="question">Question text goes here</div>
51       <div class="answers" id="answers">
52         <!-- Answer buttons will be added dynamically here -->
53       </div>
54     </div>
55     <div id="result"></div>
56   </div>
57
58   <script>
59     const quizData = [{
60       question: "What is the capital of France?",
61       options: ["Berlin", "Madrid", "Paris", "Lisbon"],
62       correct: "Paris"
63     }, {
64       question: "What is 2 + 2?",
65       options: ["3", "4", "5", "6"],
66       correct: "4"
67     }, {
68       question: "What is the largest planet in our solar system?",
69       options: ["Earth", "Mars", "Jupiter", "Saturn"],
70       correct: "Jupiter"
71     }
72   ]
73
```

The status bar at the bottom indicates the cursor is at line 18, column 9, with 4 spaces, UTF-8 encoding, CRLF line endings, and the HTML document type. The port 5500 is open, and Prettier is configured for formatting.

The screenshot shows the VS Code editor with the 'index.html' file open. The Explorer sidebar on the left shows a project named 'PORTFOLIO' with various image and CSS files. The main editor area displays the following code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>
</title>
</head>
<body>
<script>
const questionEl = document.getElementById('question');
const answerEl = document.getElementById('answers');
const resultEl = document.getElementById('result');

let currentQuestionIndex = 0;
let score = 0;

function loadQuestion() {
const currentQuestion = quizData[currentQuestionIndex];
questionEl.textContent = currentQuestion.question;
answerEl.innerHTML = '';

currentQuestion.options.forEach(option => {
const button = document.createElement('button');
button.textContent = option;
button.addEventListener('click', () => checkAnswer(option));
answerEl.appendChild(button);
});
}

function checkAnswer(selectedOption) {
const correctAnswer = quizData[currentQuestionIndex].correct;
if (selectedOption === correctAnswer) {
score++;
}
currentQuestionIndex++;

if (currentQuestionIndex < quizData.length) {
loadQuestion();
} else {
showResult();
}
}

loadQuestion();
</script>
</body>
</html>
```

The status bar at the bottom indicates 'Ln 18, Col 9', 'Spaces: 4', 'UTF-8', 'CRLF', 'HTML', 'Port: 5500', and 'Prettier'.

The screenshot shows the VS Code editor with the 'index.html' file open. The Explorer sidebar on the left shows the same project structure. The main editor area displays the following code:

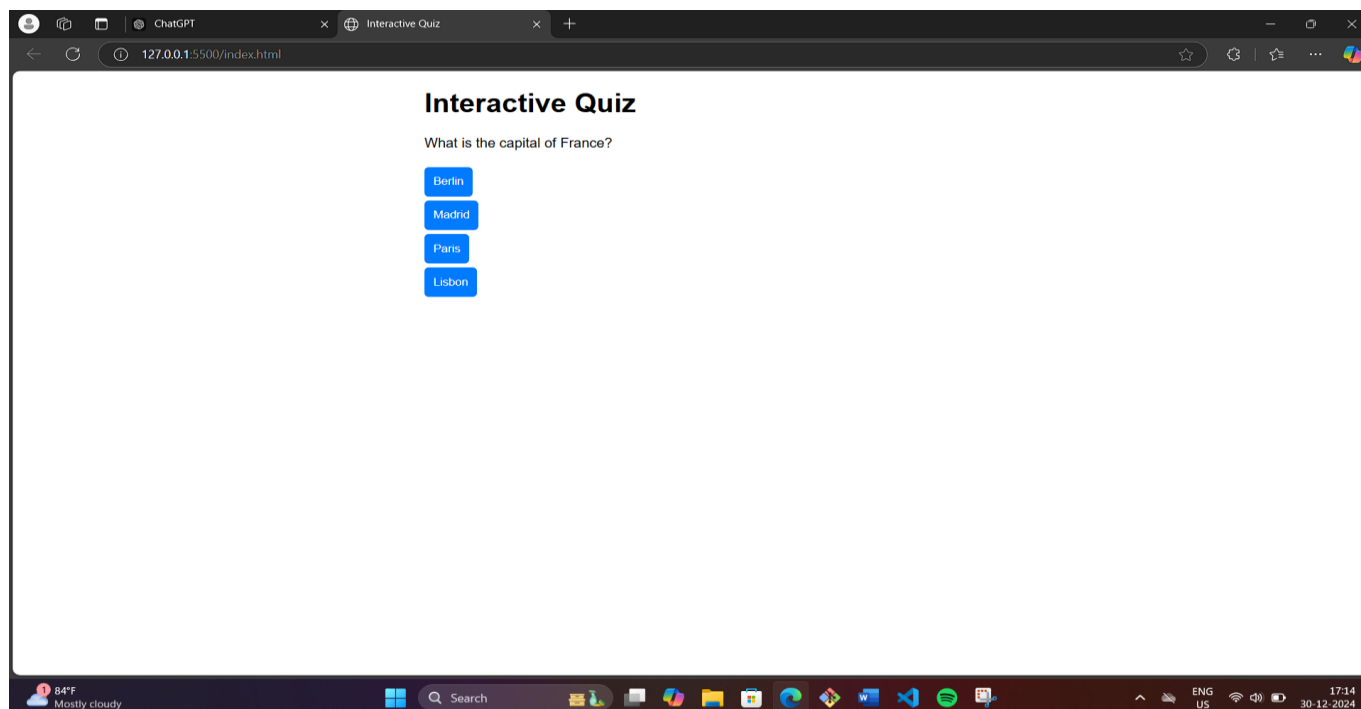
```
<!DOCTYPE html>
<html lang="en">
<head>
<title>
</title>
</head>
<body>
<script>
function checkAnswer(selectedOption) {
}

function showResult() {
questionEl.textContent = '';
answerEl.innerHTML = '';
resultEl.textContent = `You scored ${score} out of ${quizData.length}`;
}

loadQuestion();
</script>
</body>
</html>
```

The status bar at the bottom indicates 'Ln 18, Col 9', 'Spaces: 4', 'UTF-8', 'CRLF', 'HTML', 'Port: 5500', and 'Prettier'.

OUTPUT



RESULT

Thus, the quiz has been created successfully.

Ex.No : 4	SIMPLE ANIMATION USING HTML, CSS, JAVASCRIPT

AIM:

Create a simple animation using HTML, CSS and JavaScript.

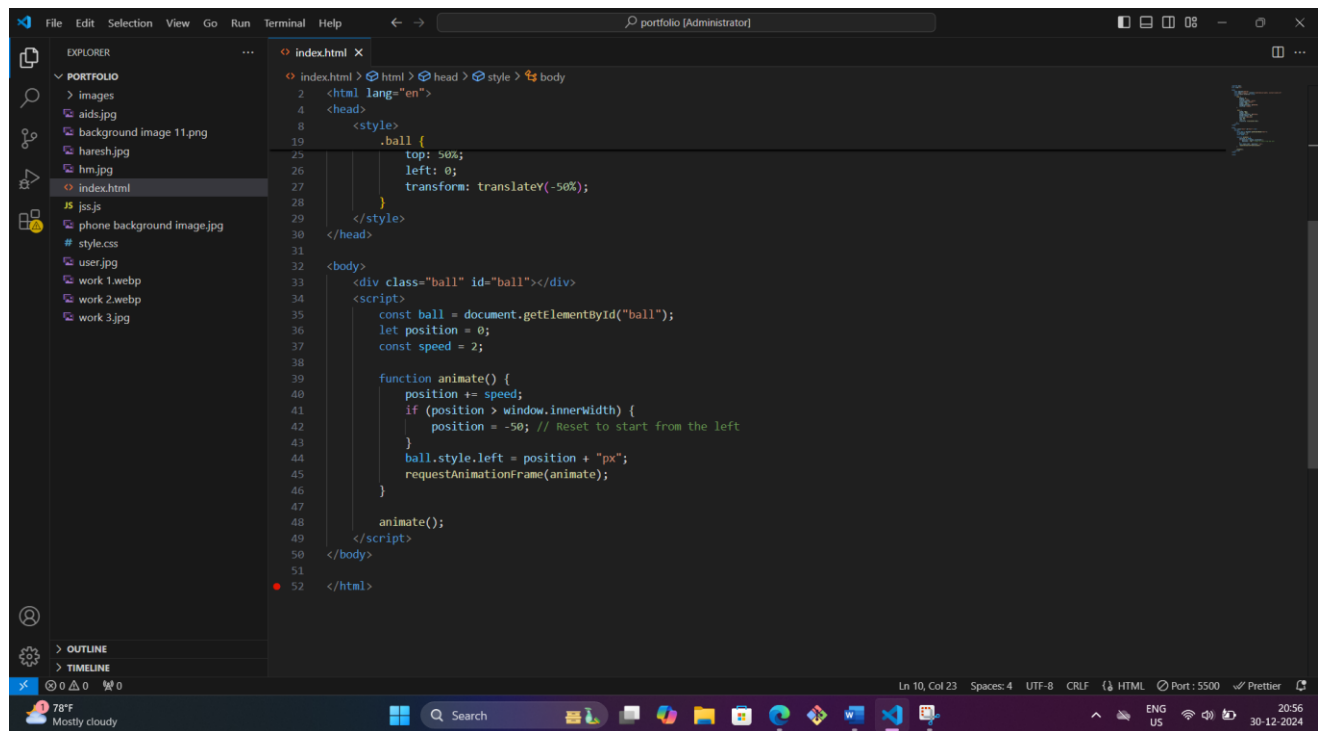
- Use CSS to define the animation's properties, such as duration and timing.
- Use JavaScript to trigger the animation in response to user input or other events

CODE

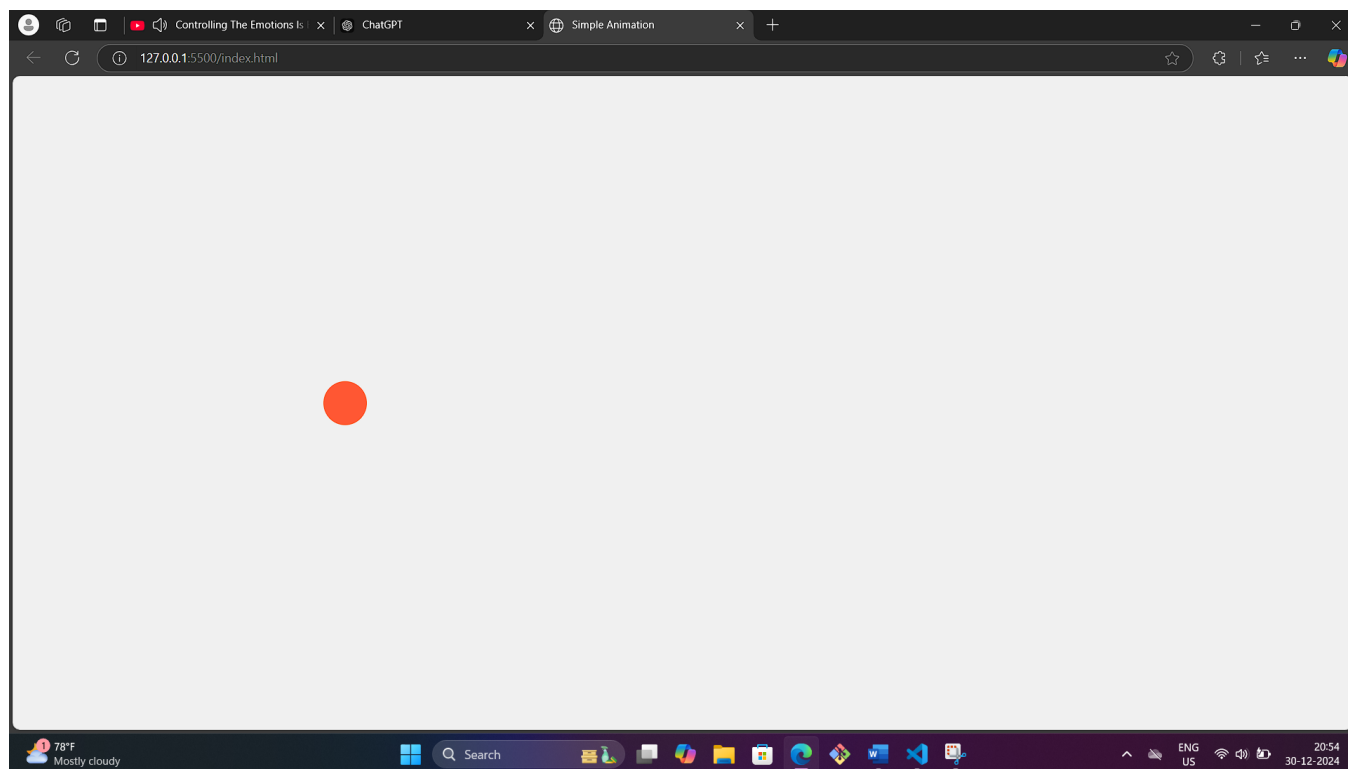
```

1  <!DOCTYPE html>
2  <html lang="en">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Simple Animation</title>
8      <style>
9          body {
10             margin: 0;
11             display: flex;
12             justify-content: center;
13             align-items: center;
14             height: 100vh;
15             background-color: #f0f0f0;
16             overflow: hidden;
17         }
18
19         .ball {
20             width: 50px;
21             height: 50px;
22             background-color: #ff5733;
23             border-radius: 50%;
24             position: absolute;
25             top: 50%;
26             left: 0;
27             transform: translateY(-50%);
28         }
29     </style>
30 </head>
31
32 <body>
33     <div class="ball" id="ball"></div>
34     <script>
35         const ball = document.getElementById("ball");
36         let position = 0;
37         const speed = 2;

```



OUTPUT



RESULT

Thus, the animation has been created successfully.

Ex.No : 5	GIT & GITHUB OPERATIONS

AIM

Write the git and GitHub commands to perform the operations for

- Setting up a local repository, ○
- Managing multiple branches, ○
- CRUD with Shell scripting.

GIT COMMANDS

Set up a local repository.

1. Setup the Working Directory for a New Project:

[Add Screenshot](#)

2. Create a new file in Git & Initialize a new Git Repository (git init)

[Add Screenshot](#)

index.html

3. Staging File Changes for Tracking (git add <file>...)

[Add Screenshot](#)

4. Committing File Changes (git commit)

[Add Screenshot](#)

5. Viewing the Commit Data (git log)

[Add Screenshot](#)

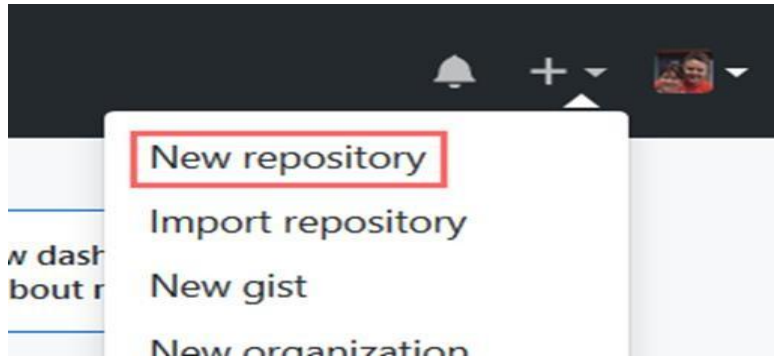
6. Viewing the difference between committed file and staged file.

- We can inspect all the unstaged changes using "git diff" command (or "git diff <file>" for the specified file). It shows the file changes in the working tree since the last commit:

[Add Screenshot](#)

7. Setting up Remote Repository

1. Sign up for a GIT host, such as GitHub <https://github.com/signup/free>
2. Login to the GIT host.
3. Create an empty remote repository in GitHub where we will push our code.
4. Click on the plus sign in the top right corner and choose New repository.



5. Enter a repository name, and an optional description, and choose Public as illustrated in Fig. Do not initialize the repository with a README or add a .gitignore file at this point, as we will do that later - along with the license. Finally, click on Create repository.

Create a new repository

A repository contains all project files, including the revision history.

Owner

 gacanepa ▾

Repository name *

/ learn-git ✓

Great repository names are short and memorable. Need inspiration? How about **glowing-rotary-phone**?

Description (optional)

Pluralsight Guide



Public

Anyone can see this repository. You choose who can commit.



Private

You choose who can see and commit to this repository.

☐ **Initialize this repository with a README**

This will let you immediately clone the repository to your computer. Skip this step if you're importing an existing repository.

Add .gitignore: **None** ▾

Add a license: **None** ▾



Create repository

6. Copy the remote repository URL and take note of the suggested commands to push the local one over to GitHub through the command line but do not run them yet.
7. On your local repo , set up the remote repo's name and URL via the "git remote add <remote-name> <remote-url>" command. By convention, we shall name our remote repo as "origin". You can find the URL of a remote repo from the Git host. The URL may take the form of HTTPS or SSH. Use HTTPS for simplicity.
8. You can list all the remote names and their corresponding URLs via "git remote -v", for example.

[Add Screenshot](#)

9. Push the commits from the local repo to the remote repo via "git push -u <remote-name> <local-branch-name>".

[Add Screenshot](#)

10. On your local system, make some changes (e.g., on "index.html"); stage and commit the changes on the local repo; and push it to the remote. This is known as the "Edit/Stage/Commit/Push" cycle.

[Add Screenshot](#)

Cloning a Project from a Remote Repo (git clone <remote-url>)

11. Anyone having read access to your remote repo can clone your project. You can also clone any project in any public remote repo. The "git clone <remote-url>" initializes a local repo and copies all files into the working tree. You can find the URL of a remote repo from the Git host.

Manage multiple branches.

- Branching allows you and your team members to work on different aspects of the software concurrently (on so-called feature branches), and merge into the master branch as and when they are complete.

1. Creating a new Branch (git branch <branch-name>)

[Add Screenshot](#)

2. Switching to a Branch (git checkout <branch-name>)

Git uses a special pointer called HEAD to keep track of the branch that you are working on. The "git branch <branch-name>" command simply create a branch, but does not switch to the new branch. To switch to a branch, use "git checkout <branch-name>" command.

[Add Screenshot](#)

[Add Screenshot](#)

[Add Screenshot](#)

3. Merging Two Branches (git merge <branch-name>)

To merge two branches, says master and devel, check out the first branch, e.g, master, (via "git checkout <branch-name>") and merge with another branch, e.g., devel, via command "git merge <branch-name>".

[Add Screenshot](#)

4. Deleting a Merged Branch (git branch -d <branch-name>)

The merged branch (e.g., devel) is no longer needed. You can delete it via "git branch -d <branchname>".

[Add Screenshot](#)

CRUD operations using Shell scripting

[Add Screenshot](#)

[Add Screenshot](#)

[Add Screenshot](#)

[Add Screenshot](#)

RESULT:

Thus, the git and GitHub commands are executed successfully and verified

Ex. No: 6	FILE MANAGEMENT USING SHELL SCRIPTING

AIM

Create a shell script that can automate file management tasks such as processing, listing, creation and handling of files and user and group management tasks

[Add Screenshot](#)

[Add Screenshot](#)

RESULT

Thus, the scripts are executed successfully and verified.

Ex.No : 7a	SIMPLE CALCULATOR -MIT APP INVENTOR

AIM

To create **Simple Calculator** mobile application using MIT APP INVENTOR

PROCEDURE

1. Access MIT App Inventor:

- Open your web browser and go to the MIT App Inventor website:
<https://appinventor.mit.edu/>.
- Sign in with your Google account.

2. Create a New Project:

- Click on "Start New Project."
- Give your project a name, and choose a project template if desired.

3. Design the User Interface:

- On the left side, you'll see the "Palette" containing different components.
Drag and drop the following components into the designer window: -
`TextBox` (for input and display)
- `Button` (for numbers and operations) - `Label` (optional, for
displaying the operation)

4. Set Component Properties:

- Click on each component in the designer window, and in the properties window, set their properties such as `Hint` for TextBox and `Text` for Button.

5. Add Event Handlers:

- Click on the "Blocks" button to switch to the Blocks Editor.
- Use the blocks to define the app's behavior. For example:
- For each number button, create an event handler that appends the corresponding number to the TextBox.
- For operation buttons (`+`, `-`, `\*`, `/`), create event handlers to perform calculations and update the TextBox.
- You can use additional variables and logic to handle the calculator's functionality.

6. Implement the Calculator Logic:

- Use the math blocks available in MIT App Inventor to perform calculations.
- Create variables to store the current input and result.
- Implement the logic to handle different operations when the corresponding buttons are pressed.

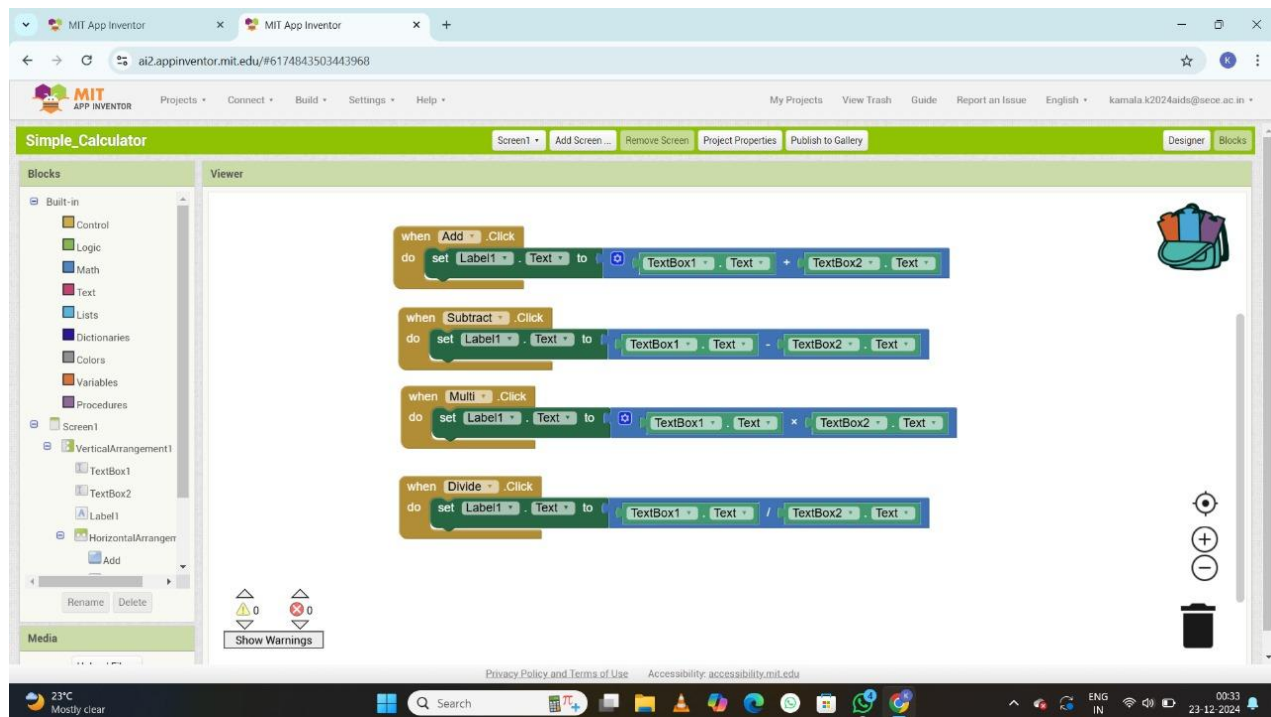
7. Testing:

- Connect your Android device to the computer or use the MIT AI2 Companion app to test your calculator in real-time.
- Check for any errors or unexpected behavior and make adjustments in the Blocks Editor.

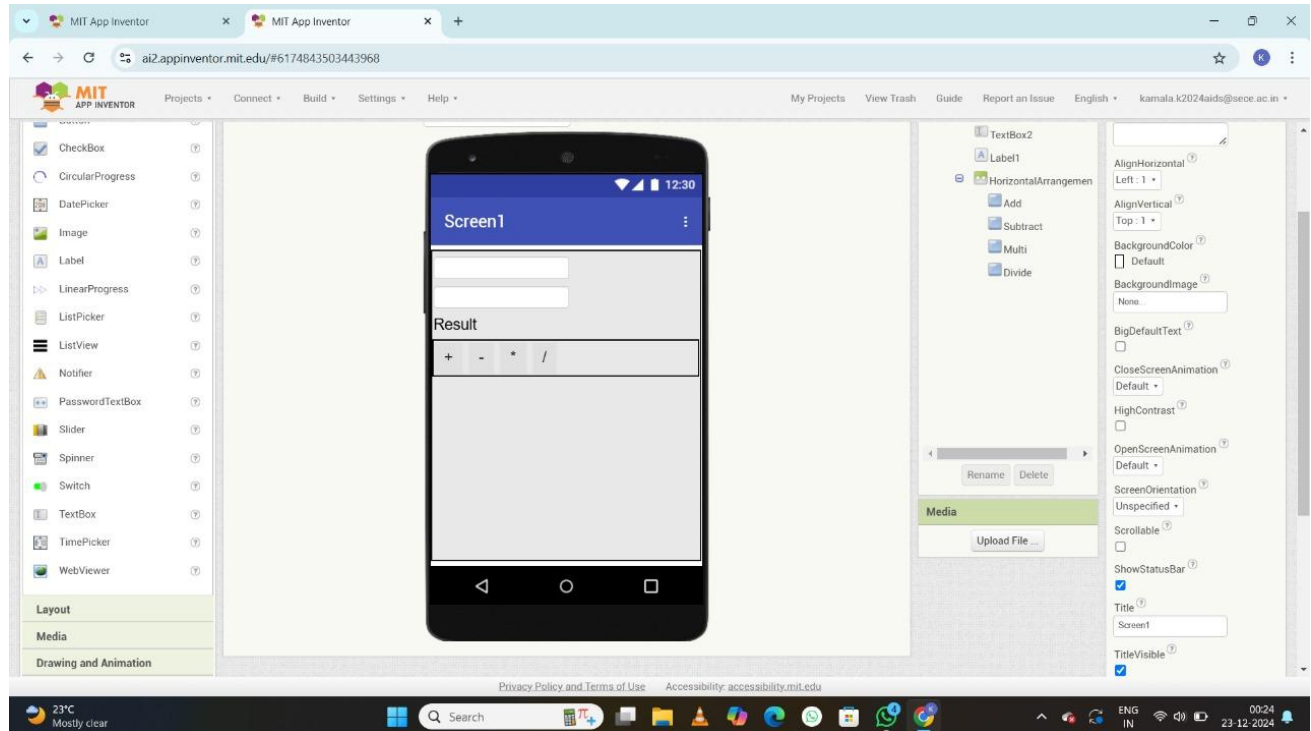
8. Build the APK:

- Once you are satisfied with your app, you can build the APK by going to "Build" and selecting "App (provide QR code for .apk)."
- Scan the QR code with the MIT AI2 Companion app, or download the APK and install it on your Android device.

CODE BLOCKS



DESIGN & OUTPUT



RESULT

Thus, the app invented successfully.

Ex.No : 7b	STEP COUNTER -MIT APP INVENTOR

AIM

To create **Step Counter** mobile application using MIT APP INVENTOR

PROCEDURE

1. Access MIT App Inventor:

- Open your web browser and go to the MIT App Inventor website:
<https://appinventor.mit.edu/>.
- Sign in with your Google account.

2. Create a New Project:

- Click on "Start New Project."
- Give your project a name, and choose a project template if desired.

3. Design the User Interface:

- On the left side, you'll see the "Palette" containing different components. Drag and drop the following components into the designer window:
- `Label` (for displaying the step count)
- `Button` (optional, for resetting the step count)
- `AccelerometerSensor` (for detecting steps)

4. Set Component Properties:

- Click on each component in the designer window, and in the properties window, set their properties. For example, set the text of the label to "Step Count: 0."

5. Add Event Handlers:

- Click on the "Blocks" button to switch to the Blocks Editor.
- Use the blocks to define the app's behavior. For example:
- When the screen initializes, you can set initial values and start the accelerometer sensor.
- Use the accelerometer sensor's `AccelerationChanged` event to detect steps.
- Update the step count label each time a step is detected.
- Optionally, use a button's click event to reset the step count.

6. Implement Step Detection Logic:

- Use variables to store the current step count and implement logic to detect steps based on changes in acceleration.
- You may need to experiment with the accelerometer sensor's sensitivity and thresholds to accurately detect steps.

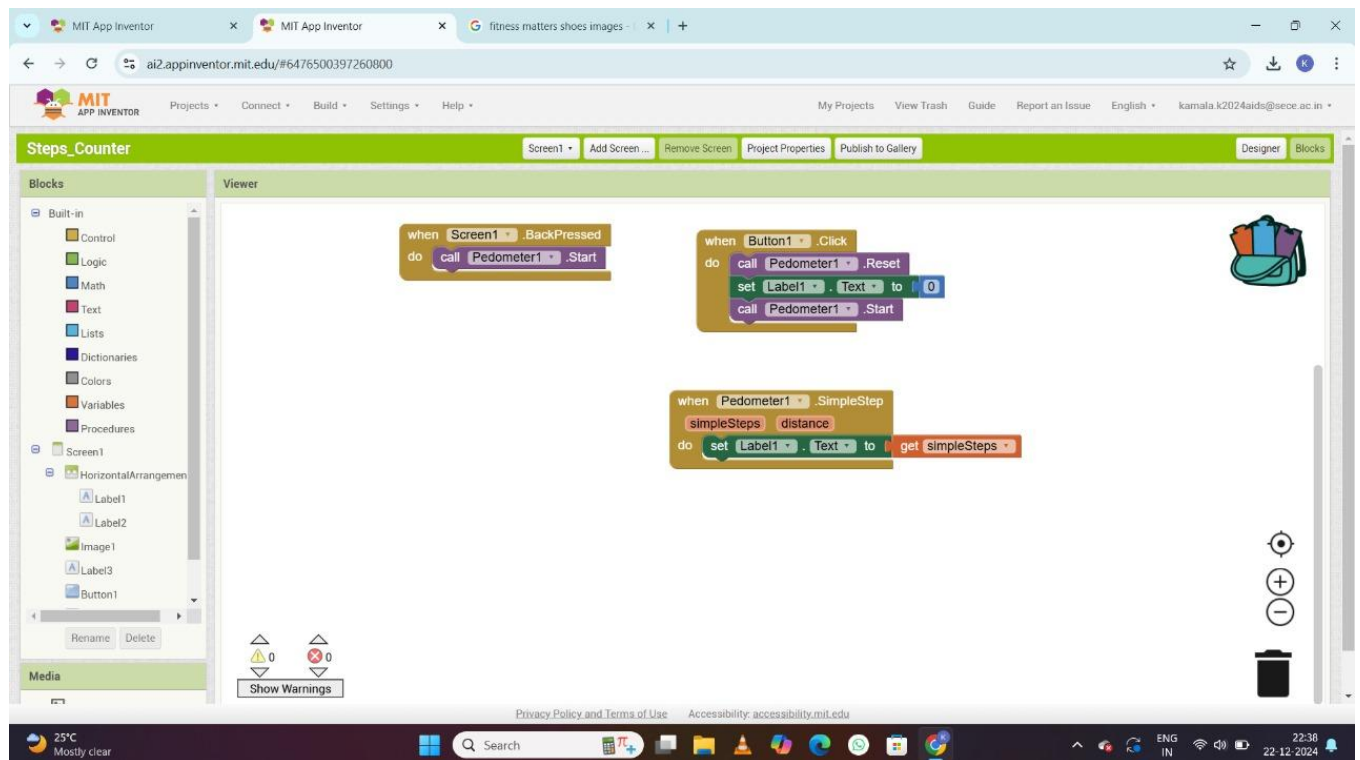
7. Testing:

- Connect your Android device to the computer or use the MIT AI2 Companion app to test your Step Counter in real-time.
- Walk or shake the device to test the step detection logic.

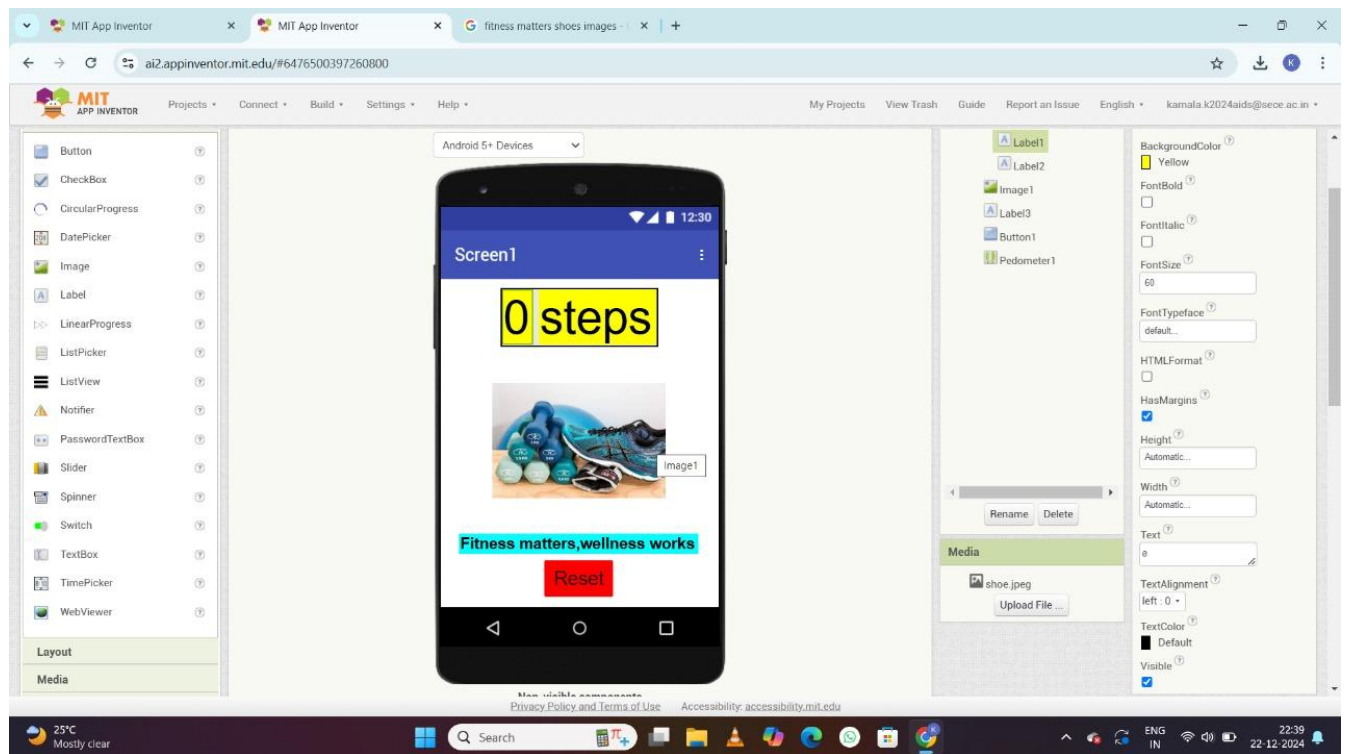
8. Build the APK:

- Once you are satisfied with your app, you can build the APK by going to "Build" and selecting "App (provide QR code for .apk)."
- Scan the QR code with the MIT AI2 Companion app or download the APK and install it on your Android device.

CODE BLOCKS



DESIGN & OUTPUT



RESULT

Thus, the mobile application created successfully.

Ex.No : 7c	WEATHER APP -MIT APP INVENTOR

AIM

To create a **Weather app** mobile application using MIT APP INVENTOR

PROCEDURE

1. Get API Key:

- Sign up for a free account on the OpenWeatherMap website:
<https://openweathermap.org/>.
- Once registered, obtain your API key.

2. Create a New Project:

- Open MIT App Inventor and start a new project.

3. Design the User Interface:

- Drag and drop the following components into the designer window:
- `TextBox` for entering the city name or zip code.
- `Button` to trigger the weather retrieval.
- `Label` to display the weather information (temperature, description, etc.).

4. Set Component Properties:

- Set appropriate properties such as 'Hint' for the TextBox and 'Text' for the Button.

5. Add Web Component:

- Drag the 'Web' component from the Palette to the designer. This will be used to make API requests.

6. Set Web Component Properties:

- Set the 'Web1' component's 'Url' property to the OpenWeatherMap API endpoint, including your API key. - Example URL:

`'https://api.openweathermap.org/data/2.5/weather?q=CityName&appid=YourAPIKey'`

7. Add Event Handlers:

- Switch to the Blocks Editor and add blocks to handle the Button's click event.
- Use the 'Web1' component blocks to make a request to the API when the Button is clicked.
- Parse the JSON response to extract the weather information.
- Update the Label to display the weather details.

8. Handle API Response:

- Use blocks from the 'Web1' component to handle the API response. You may use blocks like 'Web1.GotText' to parse the JSON response.

9. Test the App:

- Connect your device or use the MIT AI2 Companion app to test the Weather app.
- Enter a city name or zip code, click the Button, and check if the weather details are displayed.

10. Build the APK:

- Once satisfied with the app, go to "Build" and select "App (provide QR code for .apk)." - Scan the QR code with the MIT AI2 Companion app or download the APK and install it on your Android device.

CODE BLOCKS

MIT App Inventor interface showing the 'Weather_app' project. The 'Blocks' panel on the left lists components: Math, Text, Lists, Dictionaries, Colors, Variables, Procedures, and Screen1. The 'Viewer' panel on the right displays the code blocks for the app.

Code Blocks:

- Initialize global APIKey to "Oe070afe390b41398fc132548242212"
- Initialize global response to ""
- when Submit.Click do
 - set Web1.Url to join("http://api.weatherapi.com/v1/current.json?key=", global APIKey, "&q=", CityTxt.Text, "&aq=no")
 - call Web1.Get
- when Web1.GotText do
 - if get responseCode ≥ 400 then
 - set Weather_icon.Picture to "No available assets"
 - set describLbl.Text to "Data not found"
 - set tempLbl.Text to ""
 - else
 - set global response to call Web1.JsonTextDecodeWithDictionaries(jsonText, get responseContent)
 - set tempLbl.Text to ...

The bottom status bar shows the system clock: 26°C, Haze, 20:02, 22-12-2024.

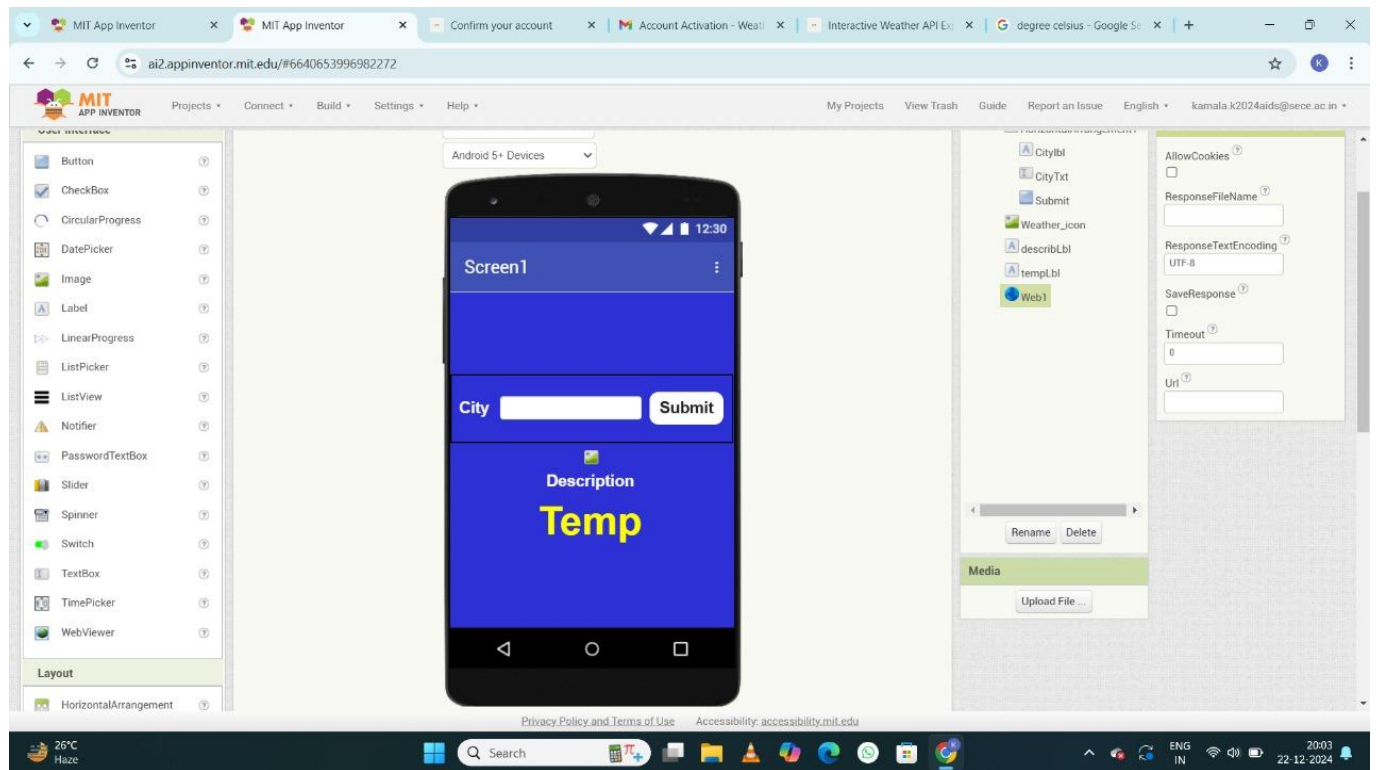
MIT App Inventor interface showing the 'Weather_app' project. The 'Blocks' panel on the left lists components: Math, Text, Lists, Dictionaries, Colors, Variables, Procedures, and Screen1. The 'Viewer' panel on the right displays the code blocks for the app.

Code Blocks:

- set tempLbl.Text to join(get value for key "temp_c" in dictionary get value for key "current" in dictionary get global response or if not found "not found")
- set describLbl.Text to join(get value for key "condition" in dictionary get value for key "current" in dictionary get global response or if not found "not found")
- set Weather_icon.Picture to join("https://", get value for key "icon" in dictionary get value for key "condition" in dictionary get global response or if not found "not found")

The bottom status bar shows the system clock: 26°C, Haze, 20:02, 22-12-2024.

DESIGN & OUTPUT



RESULT

Thus, the mobile application created successfully.

Ex. No: 7d	TALK TO ME -MIT APP INVENTOR

AIM

To create a **Talk to Me** mobile application using MIT APP INVENTOR

PROCEDURE

1. Access MIT App Inventor:

- Open your web browser and go to the MIT App Inventor website:
<https://appinventor.mit.edu/>.
- Sign in with your Google account.

2. Create a New Project:

- Click on "Start New Project."
- Give your project a name, and choose a project template if desired.

3. Design the User Interface:

- On the left side, you'll see the "Palette" containing different components. Drag and drop the following components into the designer window:
- `TextBox` for displaying the spoken text.
- `Button` for triggering speech recognition.
- `TextToSpeech` for converting text to speech (optional).

4. Set Component Properties:

- Click on each component in the designer window, and in the properties window, set their properties. For example, set the button text to "Listen."

5. Add Event Handlers:

- Click on the "Blocks" button to switch to the Blocks Editor.
- Use the blocks to define the app's behavior. For example:
- Use the `Button.Click` event to trigger speech recognition.
- Use the `SpeechRecognizer` component blocks to start listening for speech.
- Handle the `AfterGettingText` event to process the recognized speech and update the TextBox.
- Optionally, use the `TextToSpeech` component to speak a response.

6. Implement Speech Recognition:

- Drag the `SpeechRecognizer` component from the Palette to the designer.
- Set the `Language` property to the desired language.
- Use the blocks to start and stop listening for speech, and to handle the recognized text.

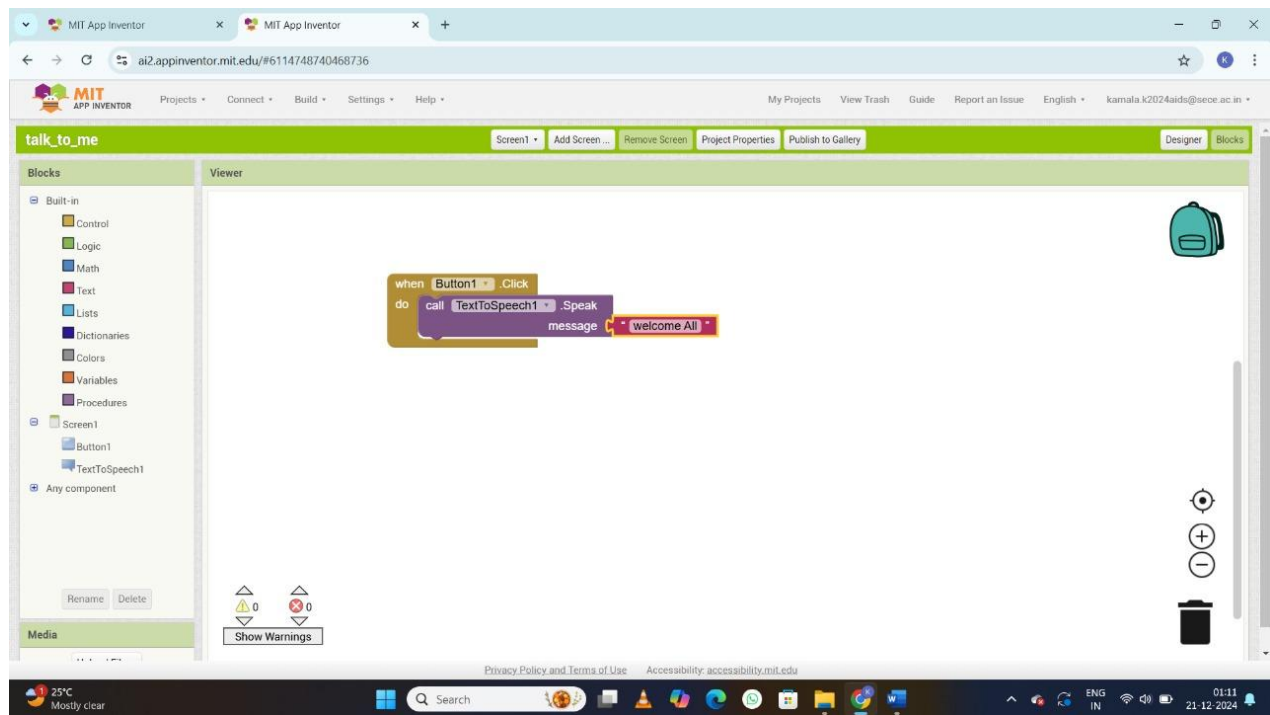
7. Test the App:

- Connect your Android device to the computer or use the MIT AI2 Companion app to test your "Talk to Me" app.
- Click the button, speak a phrase, and check if the app displays the recognized text.

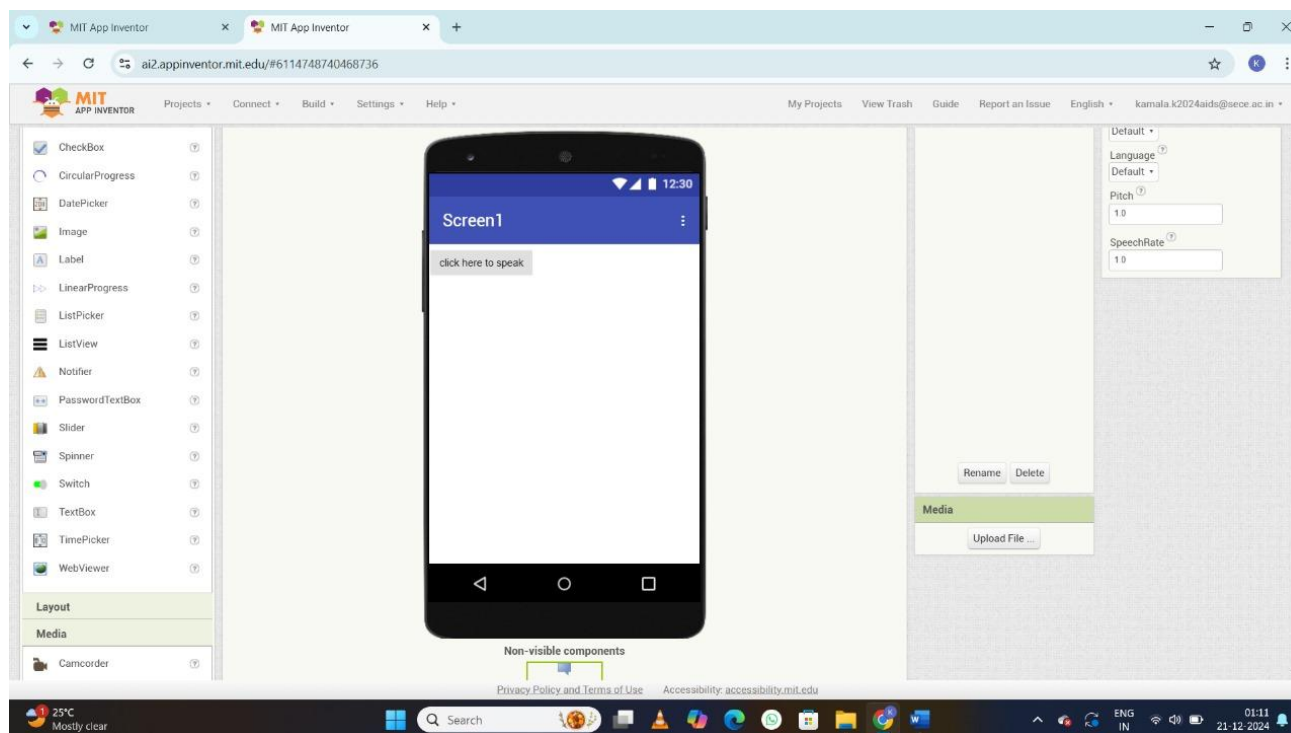
8. Build the APK:

- Once you are satisfied with your app, you can build the APK by going to "Build" and selecting "App (provide QR code for .apk)."
- Scan the QR code with the MIT AI2 Companion app or download the APK and install it on your Android device.

CODE BLOCKS



DESIGN & OUTPUT



RESULT

Thus, the mobile application created successfully.

Ex.No : 7e	TRANSLATION APP -MIT APP INVENTOR

AIM

To create a **Translation** mobile application using MIT APP INVENTOR

PROCEDURE

1. Access MIT App Inventor:

- Open your web browser and go to the MIT App Inventor website:
<https://appinventor.mit.edu/>.

- Sign in with your Google account.

2. Create a New Project:

- Click on "Start New Project."

- Give your project a name, and choose a project template if desired.

3. Design the User Interface:

- On the left side, you'll see the "Palette" containing different components. Drag and drop the following components into the designer window:

- 'TextBox' for entering text to be translated.

- `Button` to trigger the translation.
- `Label` for displaying the translated text.

4. Set Component Properties:

- Click on each component in the designer window, and in the properties window, set their properties. For example, set the button text to "Translate."

5. Add Web Component:

- Drag the `Web` component from the Palette to the designer. This will be used to make API requests.

6. Get API Key for Google Cloud Translation API:

- Go to the Google Cloud Console:
<https://console.cloud.google.com/>.
- Create a new project or use an existing one.
- Enable the "Cloud Translation API" for your project.
- Create credentials and obtain your API key.

7. Set Web Component Properties:

- Set the `Web1` component's `Url` property to the Google Cloud Translation API endpoint, including your API key.
- Example URL: `https://translation.googleapis.com/language/translate/v2?key=YourAPIKey`

8. Add Event Handlers:

- Click on the "Blocks" button to switch to the Blocks Editor.
- Use the blocks to define the app's behavior. For example:
- Use the `Button.Click` event to trigger the translation.
- Use the `Web1` component blocks to make a request to the Google Cloud Translation API.
- Handle the API response to extract and display the translated text in the Label.

9. Handle API Response:

- Use blocks from the `Web1` component to handle the API response. You may use blocks like `Web1.GotText` to parse the JSON response.

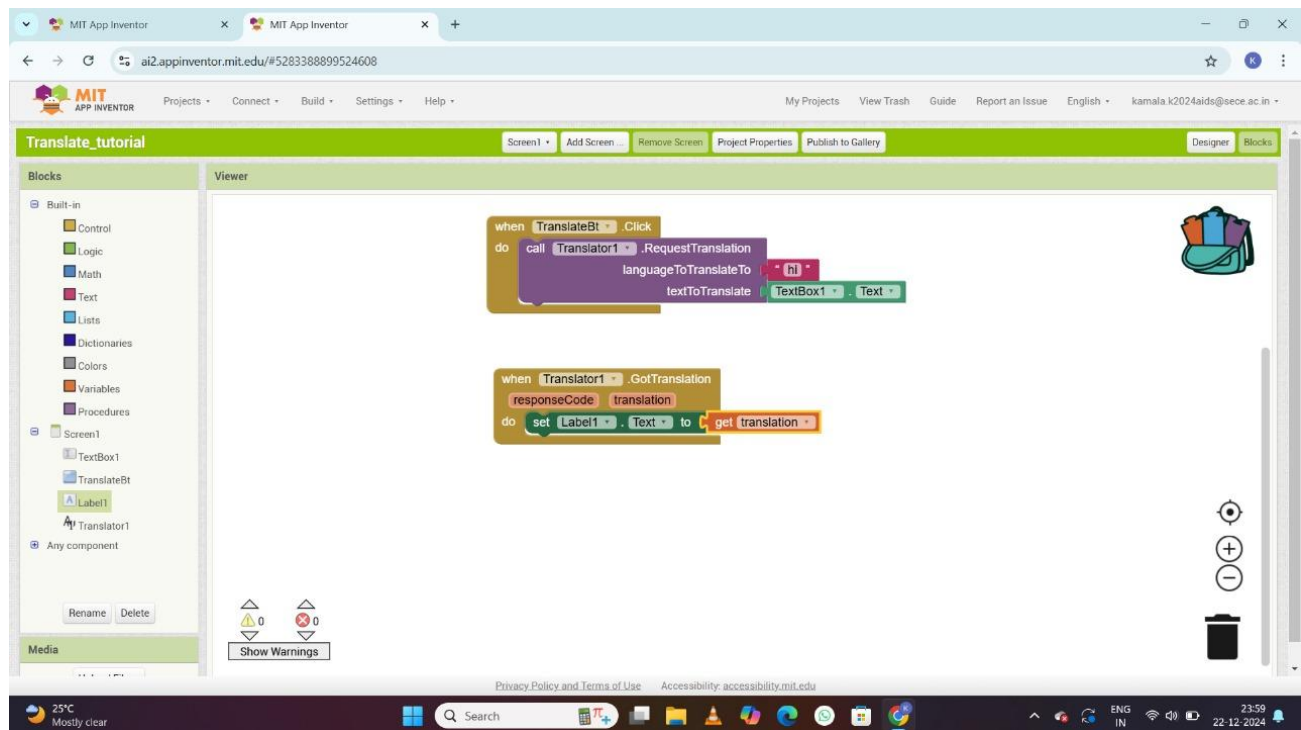
10. Test the App:

- Connect your device or use the MIT AI2 Companion app to test the Translation app.
- Enter a text, click the button, and check if the translated text is displayed.

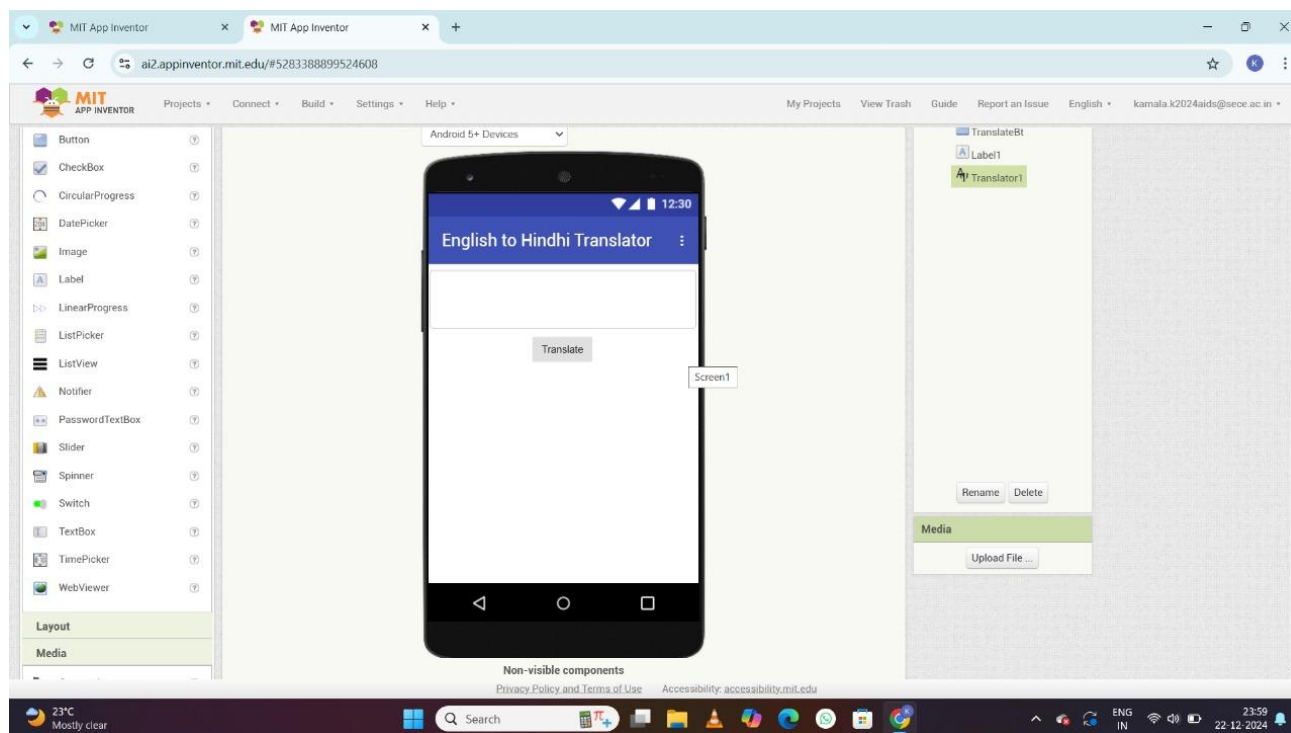
11. Build the APK:

- Once satisfied with the app, go to "Build" and select "App (provide QR code for .apk)."
- Scan the QR code with the MIT AI2 Companion app or download the APK and install it on your Android device.

CODE BLOCKS



DESIGN & OUTPUT



RESULT

Thus, the mobile application created successfully.

Ex.No : 7f	TO DO LIST -MIT APP INVENTOR

AIM

To create **To Do List** mobile application using MIT APP INVENTOR

1. Access MIT App Inventor:

- Open your web browser and go to the MIT App Inventor website:
<https://appinventor.mit.edu/>.
- Sign in with your Google account.

2. Create a New Project:

- Click on "Start New Project."
- Give your project a name, and choose a project template if desired.

3. Design the User Interface:

- On the left side, you'll see the "Palette" containing different components. Drag and drop the following components into the designer window:
 - `TextBox` for entering a new task.
 - `Button` for adding the task to the list.
 - `ListView` for displaying the list of tasks.
 - `Notifier` for displaying alerts.

4. Set Component Properties:

- Click on each component in the designer window, and in the properties window, set their properties. For example, set the button text to "Add Task."

5. Add Event Handlers:

- Click on the "Blocks" button to switch to the Blocks Editor.
- Use the blocks to define the app's behavior. For example:

- Use the `Button.Click` event to add the entered task to the list.
- Use the `ListView` blocks to update the list of tasks.
- Optionally, use the `Notifier` blocks to display alerts or notifications.

6. Implement Task List Logic:

- Use variables to store the list of tasks.
- Implement logic to add tasks to the list and update the `ListView` accordingly.
- You may also add features such as marking tasks as completed or deleting tasks.

7. Test the App:

- Connect your Android device to the computer or use the MIT AI2 Companion app to test your To-Do List app.
- Add tasks, mark them as completed, and check if the list is updated.

8. Build the APK:

- Once you are satisfied with your app, you can build the APK by going to "Build" and selecting "App (provide QR code for .apk)."
- Scan the QR code with the MIT AI2 Companion app or download the APK and install it on your Android device.

CODE BLOCKS

[Add Screenshot](#)

DESIGN & OUTPUT

[Add Screenshot](#)

RESULT

Thus, the mobile application created successfully.

Ex. No: 8	POSTER DESIGN USING CANVA

AIM:

To design a poster for any occasion using canva designer tool.

STEPS:**1. Sign in to Canva:**

- Visit [Canva](<https://www.canva.com/>).
- Sign in or create a new account.

2. Choose Poster Dimensions:

- Click on "Create a design" in the upper right corner.
- Choose "Custom dimensions" and enter the dimensions for your poster (e.g., 18x24 inches).

3. Select a Template or Start from Scratch:

- Browse Canva's template library for poster designs or start with a blank canvas.

4. Choose a Background:

- Click on the "Background" tab on the left sidebar.
- Select a color, gradient, image, or upload your own to use as the poster background.

5. Add Text:

- Click on the "Text" tab on the left sidebar.
- Choose a text style or add your own text box.
- Customize the text by changing the font, size, color, and alignment.

6. Insert Elements:

- Use the "Elements" tab to add shapes, lines, icons, or illustrations to enhance your design.
- Drag and drop elements onto your canvas.

7. Upload Images:

- If your poster requires images, click on the "Uploads" tab.
- Upload your own images or use Canva's extensive image library.

8. Arrange and Align Objects:

- Click on any element, text box, or image to select it.
- Use the toolbar at the top to arrange, align, and distribute elements evenly.

9. Apply Filters and Effects:

- Click on any image to reveal the toolbar.
- Use the "Filter" option to apply filters or adjust the transparency.

10. Add Additional Elements:

- Continue adding text, elements, or images as needed.
- Experiment with Canva's tools to enhance the visual appeal.

11. Save and Download:

- Once satisfied with your poster, click on the "Download" button in the top-right corner.
- Choose the desired file format (e.g., PDF or PNG) and click "Download."

OUTPUT



RESULT

Thus, the poster designed successfully using canva tool.

Ex.No : 9	VISUAL INFOGRAPHIC DESIGN CANVA

AIM:

To create a visual infographic using Canva tools to present data and information.

STEPS :

1. Sign in to Canva:

- Visit [Canva](<https://www.canva.com/>).
- Sign in or create a new account.

2. Choose Infographic Dimensions:

- Click on "Create a design" in the upper right corner.
- Scroll down and choose "Infographic."

3. Select a Template or Start from Scratch:

- Canva offers a variety of infographic templates. Choose one that suits your purpose or start with a blank canvas.

4. Add Sections or Blocks:

- Use the left sidebar to add sections or blocks to your infographic.
- Drag and drop them onto your canvas.

5. Edit Text:

- Click on any text box to edit the content.
- Customize the font, size, color, and alignment to suit your design.

6. Add Icons and Illustrations:

- Explore the "Elements" tab on the left sidebar.
- Add icons, illustrations, or graphics relevant to your infographic.

7. Insert Images:

- Click on the "Elements" tab and select "Images." -

Upload your own images or use Canva's image library.

8. Use Shapes and Lines:

- Add shapes or lines to create visual separation and guide the viewer's eyes.
- Customize their colors and sizes.

9. Apply Colors and Themes:

- Click on the "Colors" tab on the left sidebar to choose a color palette.
- Canva also offers pre-made themes for a cohesive design.

10. Add Charts or Graphs:

- If your information includes data, use the "Charts" tab.
- Input your data, and Canva will generate visually appealing charts.

11. Customize Layout and Spacing:

- Click on any element and use the toolbar to customize the layout.
- Adjust spacing and alignment for a clean look.

12. Add Background:

- Customize the background by clicking on the "Background" tab.
- Choose a solid color or use an image.

13. Add Annotations and Labels:

- Use text boxes or annotations to provide additional information.
- Ensure clarity and conciseness in your labels.

14. Preview and Adjust:

- Click on the "Preview" button to see how your infographic looks.
- Make any necessary adjustments for better readability.

15. Save and Download:

- Once satisfied, click on the "Download" button in the top-right corner.
- Choose your preferred file format (e.g., PDF or PNG) and download.

RESULT

Thus, the visual infographic design developed successfully using canva tool.

Ex.No : 10	GAME DEVELOPMENT USING CONSTRUCT 2

AIM:

To develop a game using construct 2 tool.

STEPS :

1. Launch Construct 2. Click the File button, and select New. You will see the 'Template or Example' dialog box.
2. The first thing we want is a repeating background tile. The Tiled Background object can do this for us. First, here's your background texture.
3. Now, double click a space in the layout to insert a new object. (Later, if it's full, you can also right-click and select Insert new object.) Once the Insert new object dialog appears, double click the Tiled Background object to insert it.
4. To manage layers, click the Layers tab, which usually is next to the Project bar, You should see Layer 0 in the list (Construct 2 counts starting from zero, since it works better

like that in programming). Click the pencil icon and rename it to Background, since it's our background layer.

5. Turn your attention back to the layout. Double click to insert another new object. This time, select the Mouse object, since we'll need mouse input. Do the same again for the Keyboard object.
6. It's time to insert our game objects! Here are your textures - save them all to disk like before.
 - Double click to insert a new object ○
Double click the 'Sprite' object.
 - When the mouse turns to a crosshair, click somewhere in the layout. The tooltip should be 'Main'. (Remember this is the active layout.)
 - The texture editor pops up. Click the open icon, and load one of the four textures.
 - Close the texture editor, saving your changes. You should now see the object in the layout!

CODE

[Add Screenshot OR CODE](#)

OUTPUT

[Add Screenshot](#)

RESULT

Thus, the above game is developed successfully using construct 2 tool.

Ex. No: 11	MAGE AND VIDEO EDITING USING BLENDER

AIM:

To edit the images and video using the blender software tool.

STEPS:

Image Editing:

1. Visit Blender.org to download the blender software package and install it in your system.
2. Choose the compositing option in the main menu to select the composite node.
3. Then import your image from the local disk and click the add menu to select the color option.
4. By using color option,the user can create the various nodes such as brightness/contrast, color correction,Hue saturation value,Mix etc.,
5. Use the viewer option in the output menu to view the image editing in background.
6. Use distort option to perform scaling , rotation ect., in the image.

7. Finally, use the render image option in the render menu to see the outcome of the edited image and save the file in your local disk or in a cloud drive.

Video Editing:

1. Choose the video editing option in blender and upload the video to be edited from local disk or cloud drive.
2. Set the timeline for video and audio in zero to ensure the synchronization in output video.
3. Click the output properties icon to set the constraints such as format, frame range, output location, audio codec etc.,
4. Finally, use the Render Animation option in the Render menu to generate the edited video and save it to your local disk or in cloud drive.

OUTPUT

Add Screenshot

RESULT

Thus, the image and video were edited successfully using the blender tool.

Ex. No: 12	SIMPLE ANIMATION USING BLENDER

AIM:

To animate a short scene or character using Blender animation tools and timeline **STEPS:**

1. **Open Blender:**
 - a. If you don't have Blender installed, download and install it from the official website: <https://www.blender.org/download/>
2. **Layout Setup:**
 - a. Open Blender and switch to the "Animation" workspace from the top menu.
3. **Import or Create Your Scene:**
 - a. Either import a 3D model or create your scene using Blender's modeling tools.
4. **Set Up Armature (if animating a character):**
 - a. If you're animating a character, create an armature (skeleton) using bones.
 - b. Place bones in key areas like the hips, shoulders, and joints.
5. **Bind Mesh to Armature:**
 - a. Select your 3D model, then shift-select the armature.
 - b. Press **Ctrl + P** to parent the mesh to the armature and choose "With Automatic Weights."
6. **Pose Mode:**
 - a. Switch to Pose Mode for the armature.
 - b. Pose your character by rotating and moving the bones.
7. **Set Keyframes:**

- a. Move to the first frame on the timeline (default: frame 1).
 - b. Select the bones or objects you want to animate.
 - c. Press **I** and choose the type of keyframe (Location, Rotation, Scale).
- 8. Move to Another Frame:**
 - a. Move to a different frame on the timeline.
 - b. Adjust the pose or position of the objects.
 - c. Set another keyframe.
- 9. Repeat and Refine:**
 - a. Continue moving to different frames, adjusting poses, and setting keyframes.
 - b. Blender will interpolate between keyframes to create smooth animations.
- 10. Timeline and Playback:**
 - a. Use the timeline at the bottom to navigate frames.
 - b. Press **Spacebar** to play the animation and check for smoothness.
- 11. Graph Editor (Optional):**
 - a. For more advanced control, use the Graph Editor to fine-tune animation curves.
- 12. Camera Animation (Optional):**
 - a. If you have a scene, animate the camera by adding keyframes for its position and rotation.
- 13. Rendering:**
 - a. Once satisfied, go to the "Render" workspace.
 - b. Set your output settings (resolution, format, etc.).
 - c. Click "Render Animation" to create the final animation.
- 14. Save and Export:**
 - a. Save your Blender project (**File > Save**).
 - b. Export your animation in a desired format (**File > Export > FBX** for compatibility with other software).
- 15. External Rendering (Optional):**
 - a. For higher-quality rendering, you might consider using external render engines like Cycles or Eevee.

OUTPUT

Add Screenshot

RESULT

Thus, the animate a short scene or character using Blender animation tools and timeline is successfully completed.